Sistema de inscripciones

Arquitectura, autenticación JWT y reproducción de la entrega

El Hito 2 incorpora identidad autenticada al sistema del Hito 1. Se mantienen Django, PostgreSQL y Docker Compose. El frontend activo usa Astro para renderizar páginas y componentes React para login, sesión e inscripciones. El entorno de entrega agrega Nginx y Gunicorn.

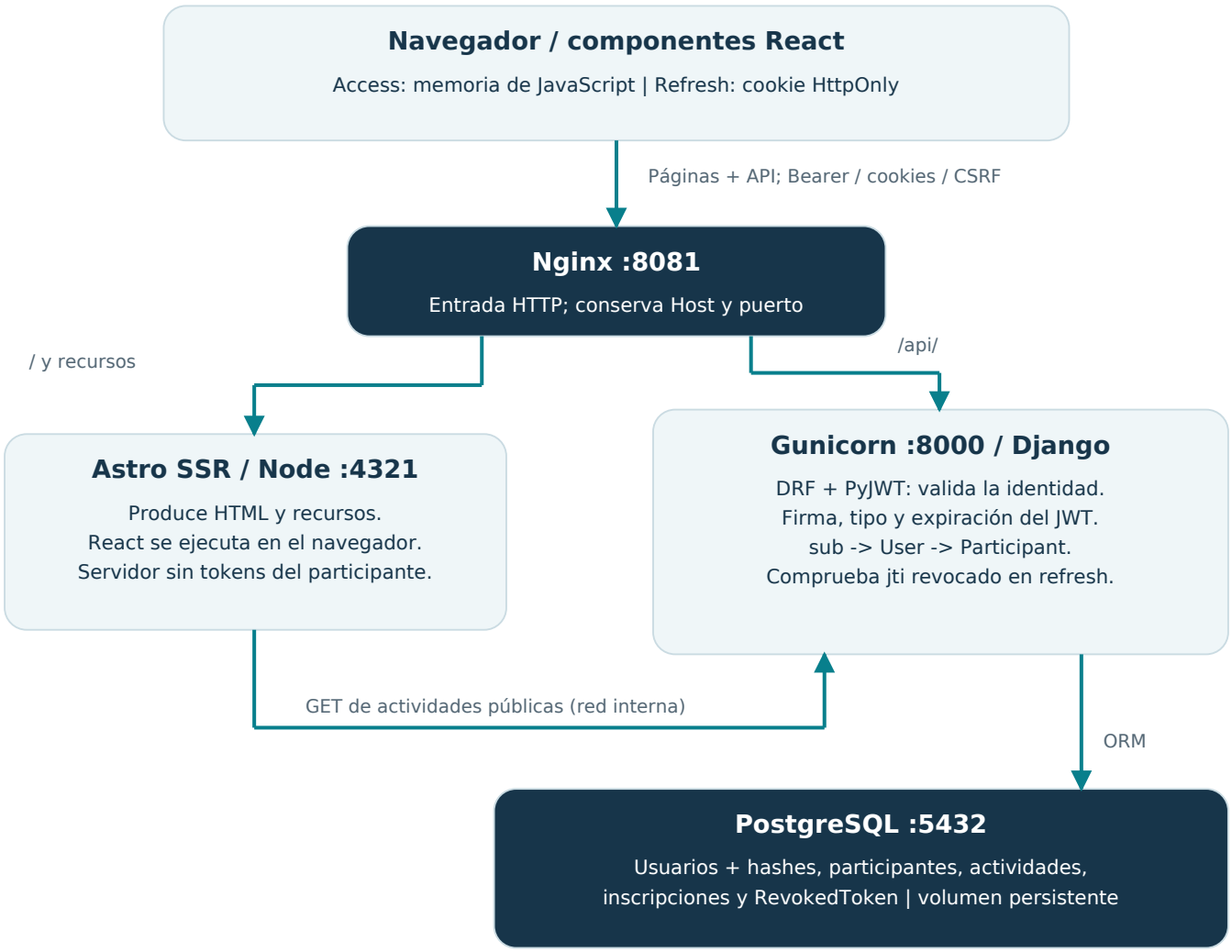

Diagrama del entorno de entrega. Las flechas indican solicitudes; las respuestas vuelven por el mismo recorrido.

Entrada, datos e identidad

Nginx publica sólo el puerto local 8081: envía **/api/** a Django y las demás rutas a Astro. Astro consulta actividades públicas al backend por la red interna de Compose. Los tokens del participante se manejan en el navegador, no durante ese renderizado del servidor.

Django valida identidad y reglas del dominio. **PostgreSQL** conserva usuarios y hashes de contraseña, participantes, actividades, inscripciones y revocaciones. El volumen **postgres_data** conserva esos datos al recrear contenedores.

Autenticación y autorización

PyJWT genera y valida los JWT con HS256. No se utiliza SimpleJWT. Los JWT están firmados: sus claims son legibles y no se usan para guardar contraseñas.

Token	Contenido y duración	Ubicación y uso
Access	sub, username, type=access, iat, exp. **10 minutos.**	Sólo memoria del frontend. Authorization: Bearer <token>
Refresh	sub, type=refresh, jti único, iat, exp. **7 días.**	Cookie refresh_token: HttpOnly, SameSite=Lax, Path=/api/v2/auth/.

1. Login

El cliente envía username y password a **POST /api/v2/auth/login/**. Django llama a **authenticate()**, comprueba que el usuario tenga participante y emite ambos tokens. El access vuelve en JSON; el refresh se instala como cookie HttpOnly. Una contraseña incorrecta devuelve **401**; un usuario válido sin participante recibe **403**.

2. Operaciones del participante

En las rutas **/api/v2/me/**, AccessTokenAuthentication comprueba firma, algoritmo permitido, claims obligatorios, expiración y type=access. Busca un usuario activo usando sub; luego obtiene **request.user.participant**. La identidad no se toma de un UUID elegido por el cliente. Un access faltante, vencido o inválido devuelve **401**; la ausencia de participante, **403**.

3. Renovación

POST /api/v2/auth/refresh/ recibe la cookie y valida firma, expiración, type=refresh y jti. Rechaza identificadores presentes en **RevokedToken** y comprueba usuario activo y participante. Si corresponde, entrega un nuevo access. El refresh mantiene su vencimiento original; no se rota. Un refresh ausente, inválido, vencido o revocado devuelve **401**.

4. Logout

POST /api/v2/auth/logout/ registra el jti del refresh válido, elimina su cookie y devuelve **204**. Si ya no hay un refresh válido, también responde 204. El frontend descarta el access y limpia la información privada. La revocación queda persistida en PostgreSQL.

CSRF y cookies

Antes de los tres POST de autenticación, el cliente consulta **GET /api/v2/auth/csrf/**, conserva las cookies y envía el valor csrf_token en **X-CSRFToken**. Una verificación CSRF fallida devuelve 403. El navegador accede al mismo origen mediante Nginx; conservar Host y el puerto externo evita rechazos de origen. La evaluación usa HTTP local, por eso Secure está desactivado.

Decisiones y límites

Conservación del Hito 1

Operación v2	Comportamiento
GET activities/ y activities/{id}/	Consulta pública de actividades y cupos. Detalle inexistente: 404.
GET me/enrollments/	Sólo las inscripciones del participante autenticado.
PUT me/enrollments/create/{id}/	201 al crear; 200 si ya existe; 409 sin cupo; 404 si no existe la actividad. No se envía participante.
DELETE me/enrollments/delete/{id}/	204 al cancelar o si ya estaba cancelada; 404 si no existe la actividad.

Access breve y refresh HttpOnly

El access se conserva en una variable del módulo de autenticación; no se escribe en localStorage, sessionStorage ni cookies. Al recargar o cambiar de página Astro se pierde esa memoria y se recupera un access por refresh. Ante un 401 protegido, el cliente intenta renovar y reintenta una vez. Comparte las renovaciones simultáneas y evita restaurar una sesión desde respuestas anteriores al logout.

HttpOnly evita que JavaScript lea el refresh, pero no impide que un script malicioso haga peticiones dentro de una sesión. CSRF y la prevención de XSS siguen siendo necesarios. La cookie se restringe a las rutas de autenticación.

Qué pasa con un access emitido después del logout

El logout revoca el refresh presentado, no el access. Si alguien conserva una copia del access, puede utilizarla hasta su expiración original, mientras el usuario siga activo y tenga participante. Por ejemplo: un access emitido a las 10:00 vence a las 10:10; un logout a las 10:03 no elimina los 7 minutos restantes. La corta duración limita esa ventana.

Límites de la implementación

Sesiones: no hay rotación del refresh, detección de reutilización ni cierre global de sesiones. El logout afecta a su jti. No hay limpieza automática de revocaciones vencidas ni limitación de intentos de login.

API heredada: /api/v1/ permanece accesible con el comportamiento anterior y sin JWT. La interfaz actual usa v2. Esta permanencia impide afirmar que todas las rutas están protegidas; se conserva como una limitación explícita del alcance.

Concurrencia: cupo e idempotencia se comprueban para operaciones secuenciales. La unicidad actividad-participante evita filas duplicadas, pero el conteo y la creación no están bloqueados en una transacción: solicitudes simultáneas pueden superar el cupo o generar un error en una carrera por la misma inscripción.

Evaluación local: HTTP sin TLS, clave Django/JWT fija en código y cuentas públicas de prueba. No se presenta como una configuración segura para Internet. Las etiquetas de imágenes y algunos rangos Python tampoco garantizan builds idénticos bit a bit en fechas distintas.

Ejecución y comprobación

Repositorio: github.com/jeretamargo/actividad_2-Arquitecturas_Web. El **README de la raíz** contiene los comandos completos, requisitos, puertos y resolución del entorno local. No depende de archivos .env privados ni de una base preexistente.

Desde una copia limpia

Paso	Acción
1. Configuración	Clonar el repositorio. Copiar .env.example como .env: DB_NAME, DB_USER y DB_PASSWORD públicos de evaluación.
2. Construcción	Usar compose.yaml + compose.prod.yaml y el nombre de proyecto hito2. Construir backend y frontend e iniciar PostgreSQL.
3. Esquema y datos	En backend: manage.py migrate, seed_activities y seed_users, en ese orden. No ejecutar makemigrations para instalar.
4. Arranque	Iniciar todos los servicios y abrir http://127.0.0.1:8081/. Nginx comunica con Astro y Gunicorn; sólo Nginx publica un puerto.

Usuarios exclusivamente locales de evaluación

Usuario	Contraseña	Participante	Inscripciones
maria	MariaTP2026!	María López	2
jorge	JorgeTP2026!	Jorge Fernández	1

seed_activities restaura 4 actividades, 5 participantes y 8 inscripciones; elimina datos de dominio ajenos al escenario. **seed_users** crea o restaura las dos cuentas sin duplicarlas ni modificar inscripciones: conserva el hash si la contraseña coincide y restaura contraseña/estado activo si cambiaron. Ante cuentas administrativas o vínculos incompatibles, revierte toda la operación.

Las migraciones versionadas incluyen la relación User-Participant (0004) y RevokedToken (0005). PostgreSQL almacena el esquema, los datos y las revocaciones en un volumen. Detener con down lo conserva; down -v lo elimina. Desarrollo usa Astro :4321 y Django :8000, sin Nginx ni Gunicorn; no sustituye la prueba del entorno de entrega.

Evidencia de verificación local

Backend: 6 pruebas aprobadas en una base de tests: listado público, método HTTP, bootstrap, login con CSRF, cookie HttpOnly, identidad autenticada, repetición de semillas, restauración de credenciales y rollback por conflicto. La comprobación de migraciones no detectó cambios pendientes.

Integración HTTP real: instalación con volumen PostgreSQL nuevo y 24 comprobaciones por Nginx: páginas SSR, login correcto/incorrecto, falta de access, CSRF, inscripciones propias, cupo 409, PUT y DELETE repetidos, refresh, borrado de cookie, revocación y vigencia del access después del logout. Las pruebas se realizaron en un entorno separado de la base de desarrollo.

Frontend: build de producción correcto y pruebas del módulo de autenticación con respuestas simuladas. Estas últimas verifican lógica del cliente; no se presentan como prueba de concurrencia ni sustituyen las comprobaciones HTTP reales.